

Branching Policy: 'Git/Gitflow' based versioning

Patrick TOCA - D&T EMEA

20 December 2018

This document describes the branching policy of the EMEA EDP team, and presents also how to use it on a day to day basis.

The objectives of this document are:

1. to describe the fundamentals principles of the EMEA EDP versioning process,
2. to present the added values presented by this Git/Gitflow based process,
3. to provides sufficient guidelines to help everyone in the team to get started and become proficient in short time,

This handout is aimed at the members of the EMEA EDP team(s) and has been distributed before the training session on Git/Gitflow held on January 2019.

Outline of the hand-out

- 1 Advantages of the Git/Gitflow based process
- 2 General concepts of versioning with Git/Gitflow
- 3 Branching terminology and what it implies
- 4 Gitflow commands in practice
- 5 Conclusion
- 6 Useful links

Advantages of the Git/Gitflow based process

Git/Gitflow facilitates the realisation of a simple working process within the development team.

It enforces certain practices such as naming conventions for your branches and assists with the flow from development, to release, to master.

Git/Gitflow doesn't offer anything that basic git commands can't do.

With the 'start' and 'finish' commands Gitflow only abstracts away the merging and deleting Git native commands, a simplification.

Benefits are:

- enforces a process (flow) by having known branch types: master, develop, feature, release, hotfix
 - standardises branch naming conventions
 - abstracts certain git commands for merging and deleting (See link here: [Git and Gitflow commands comparison: a simplification](#))
 - standardises our branching policy on validated practices.
-

General concepts of versioning with Git/Gitflow)

The document: [Git the essential concepts](#) is presenting in details what we introduce here.

Git stores and thinks about information in a very different way than other VCS (subversion and etc...), and understanding these differences will help you avoid becoming confused while using it.

A - Snapshots, Not Differences

Git thinks of its data more like a series of snapshots of a miniature filesystem. With Git, every time you commit, it saves the state of your project.

To be efficient, if files have not changed, Git doesn't store the file again, just a link to the previous identical file it has already stored. Git thinks about its data more like a stream of snapshots.

It makes Git reconsider almost every aspect of version control that most other systems copied from the previous generation.

Git has been one of the very first **fully distributed application** ever delivered in production on a large scale.

NEED TO KNOW: (see **nota bene** ¹) Version Control System provides facilities to model files changing over time and distribute different versions efficiently.

The popular version control system Git provides a powerful Merkle DAG object model that captures changes to a filesystem tree in a distributed-friendly way.

1. **Immutable objects** represent Files (blob), Directories(tree), and Changes (commit).
 2. Objects are **content-addressed**, by the cryptographic hash of their contents.
 3. Links to other objects are embedded, forming a **MerkleDAG**. This provides many useful integrity and work-flow properties.
 4. Most versioning **metadata** (branches, tags, etc.) **are simply pointer reference**, and thus inexpensive to create and update.
 5. **Version changes** only update references or add objects.
 6. **Distributing version changes** to other users is simply transferring objects and updating remote references.
-

¹ Git one of the first fully distributed, highly concurrent and scaleable application by design.

B - Nearly all operations are local

Most operations in Git need only local files and resources to operate generally no information is needed from another computer on your network. Because you have the entire history of the project right there on your local disk, most operations seem almost instantaneous.

This also means that there is very little you can't do if you're offline or off VPN.

C - Git has Integrity

Everything in Git is checksummed before it is stored and is then referred to by that checksum. This means it's impossible to change the contents of any file or directory without Git knowing about it.

This functionality is built into Git at the lowest levels and is integral to its philosophy.

The mechanism that Git uses for this checksumming is called a SHA-1 hash.

A SHA-1 hash looks something like this:

24b9da6552252987aa493b52f8696cd6d3b00373

In fact, Git stores everything in its database not by file name but by the hash value of its contents.

D - Git generally only adds data

When you do actions in Git, nearly all of them only add data to the Git database.

It is hard to get the system to do anything that is not undoable or to make it erase data in any way.

See here: [To go further on Undoing things with Git.](#)

E - Git manages 3 states

Pay attention now!

Here is the main thing to remember about Git if you want the rest of your learning process to go smoothly.

Git has three main states that your files can reside in: **committed**, **modified**, and **staged**:

- **Committed** means that the data is safely stored in your local database.
- **Modified** means that you have changed the file but have not committed it to your database yet.
- **Staged** means that you have marked a modified file in its current version to go into your next commit snapshot.

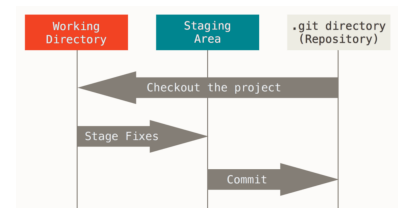


Figure 1: The 3 Git stages - Working tree, staging and Git directory

Branching terminology and what it implies

The diagram below, presents 4 important policies that Gitflow facilitates.

They organise the discipline around the Software Process:

1. the Branching policy: Feature, Develop, Release, Hotfixes (if any), Master
2. the Testing policy: developers can manage features and develop, the QA manage Release and the Owner manage Master.
3. the DB policy: a DB and its necessary and sufficient data, for each type of testing,
4. the Deployment policy: which environment with which Owner and which purpose.

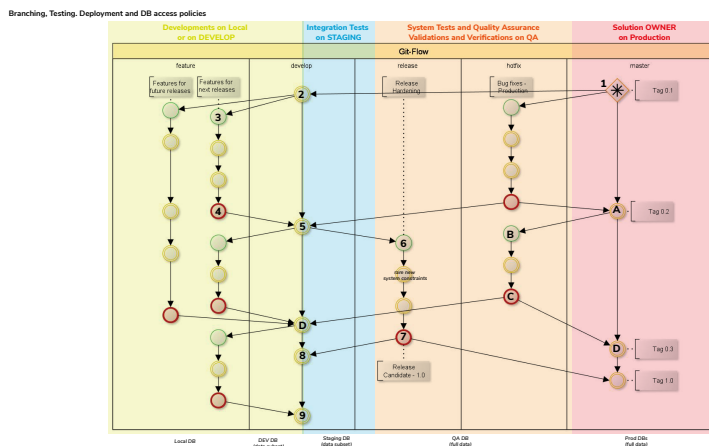


Figure 2: Branching, Testing, DB and Deployment Policies.

Gitflow commands in practice

Gitflow simplifies the Git versioning branching process to a few set of operations (see **nota bene**) ²: Initialise, Connect, Features, Releases and Hotfix.

² Using the 'showcommands' switch you can see the actual git commands being executed by the Gitflow plugin :
 "git flow feature start FEATURE-NAME -showcommands"

A - Initialise

Gitflow	Git
git flow init	git init git commit -allow-empty -m "Initial commit" git checkout -b develop master

git flow init also edits your .git/config and prompts for values:

[gitflow "branch"]

- master = master
- develop = develop

[gitflow "prefix"]

- feature = feature
 - release = release
 - hotfix = hotfix
 - support = support
 - versiontag =
-

B - Connect to Remote repository

Gitflow	Git
Already done during the init	git remote add origin git@github.com:MYACCOUNT/MYREPO

C - Features branching

Creating a feature branch

Gitflow	Git
<code>git flow feature start MYFEATURE</code>	<code>git checkout -b feature/MYFEATURE develop</code>

Sharing a feature branch

Gitflow	Git
<code>git flow feature publish MYFEATURE</code>	<code>git checkout feature/MYFEATURE</code> <code>git push origin feature/MYFEATURE</code>

Get latest for a feature branch

Gitflow	Git
<code>git flow feature pull origin MYFEATURE</code>	<code>git checkout feature/MYFEATURE</code> <code>git pull --rebase origin feature/MYFEATURE</code>

Finalize a feature branch

Gitflow	Git
<code>git flow feature finish MYFEATURE</code>	<code>git checkout develop</code> <code>git merge --no-ff feature/MYFEATURE</code> <code>git branch -d feature/MYFEATURE</code>

Push the merged feature branch

Gitflow	Git
Already done during the finish	git push origin develop git push origin :feature/MYFEATURE (if pushed)

D - Releases branching**Create a release branch**

Gitflow	Git
git flow release start v1.2.0	git checkout -b release/v1.2.0 develop

Share a release branch

Gitflow	Git
git flow release publish v1.2.0	git checkout -b release/v1.2.0 develop

Get latest for a release branch

Gitflow	Git
Already done during the finish	git checkout -b release/v1.2.0 develop git pull --rebase origin release/v1.2.0

Finalize a release branch

Gitflow	Git
git flow release finish v1.2.0	git checkout master git pull --rebase origin release/v1.2.0 git merge --no-ff release/v1.2.0 git tag -a v1.2.0 \$RELEASE_BRANCH (to include the branch name) git checkout develop git merge --no-ff release/v1.2.0 git branch -d release/v1.2.0

Push the merged release branch

Gitflow	Git
Already done during the finish	git push origin master git push origin develop git push origin --tags git push origin :release/v1.2.0 (if pushed)

E - Hotfixes branching**Create a hotfix branch**

Gitflow	Git
git flow hotfix start v1.2.1 [commit]	git checkout -b hotfix/v1.2.1 [commit]

Finalize a hotfix branch

Gitflow	Git
git flow hotfix finish v1.2.1	git checkout master git merge --no-ff hotfix/v1.2.1 git tag -a v1.2.1 git checkout develop git merge --no-ff hotfix/v1.2.1 git branch -d hotfix/v1.2.1

Push the merged hotfix branch

Gitflow	Git
Already done during the finish	git push origin master git push origin develop git push origin -tags git push origin :hotfix/v1.2.1 (if pushed)

Conclusion

Git-Flow is an extremely powerful workflow for doing development and seems to cater for just about every scenario you can come across in business.

It is simple to follow once you start using it and makes for a good standard across small and big companies.

If you use it in conjunction with Git you'll quickly realize the strength of it (combined with Test Driven Development practices, Continuous Integration as well as Regression Testing scenarios) while not even taking into consideration the usual effort that goes into doing a Release or even a Hotfix.

A - Parallel Development

One of the great things about GitFlow is that it makes parallel development very easy, by isolating new development from finished work.

New development (such as features and non-emergency bug fixes) is done in feature branches, and is only merged back into main body of code when the developer(s) is happy that the code is ready for release.

Although interruptions are a BadThing(tm), if you are asked to switch from one task to another, all you need to do is commit your changes and then create a new feature branch for your new task.

When that task is done, just checkout your original feature branch and you can continue where you left off.

B - Collaboration

Feature branches also make it easier for two or more developers to collaborate on the same feature, because each feature branch is a

sandbox where the only changes are the changes necessary to get the new feature working.

That makes it very easy to see and follow what each collaborator is doing.

C - Release Staging Area

As new development is completed, it gets merged back into the develop branch, which is a staging area for all completed features that haven't yet been released.

So when the next release is branched off of develop, it will automatically contain all of the new stuff that has been finished.

D - Support For Emergency Fixes

GitFlow supports hotfix branches - branches made from a tagged release.

You can use these to make an emergency change, safe in the knowledge that the hotfix will only contain your emergency fix.

There's no risk that you'll accidentally merge in new development at the same time.

Useful links

1. [a graphic representation of the Gitflow process](#)
2. [Vincent Driessen's original blog post about Gitflow - 2010](#)
3. [Gitflow for Github: further consolidation](#)
4. [Gitflow animation](#)
5. [Finally, to install Gitflow on your laptop](#)